Generador de tablas LR(1)

Compiladores

CS342 · UCSP · 2026-II

Introducción

Se construirá una herramienta en C++17 que, dada *cualquier* gramática libre de contexto, genere la colección canónica de ítems LR(1) y las tablas `Action` y `Goto`.

La herramienta no puede depender de la gramática de ChocoPy, todo sale del archivo de entrada. No se permite usar Bison ni ninguna biblioteca que construya autómatas o tablas.

Generador de tablas

Entrada

Un archivo con la gramática. El formato lo decide cada equipo, siempre que permita escribir cualquier gramática libre de contexto, incluidas las producciones vacías, y que identifique el símbolo inicial. Como ejemplo se usará la gramática de paréntesis.

$$\begin{array}{rcl} Goal & \to & List \\ List & \to & List\ Pair \mid Pair \\ Pair & \to & (\ Pair\ ) \mid (\ ) \end{array}$$

El símbolo `eof` es reservado y lo agrega la herramienta.

Funciones obligatorias

1. `first()` calcula los conjuntos FIRST de todos los símbolos, por punto fijo, incluyendo la marca de ε para no terminales anulables.
2. `firstCadena(`β`)` calcula FIRST de una cadena de símbolos $\beta = s_1 s_2 \dots s_k$. Es la unión de FIRST$(s_i) - \{\varepsilon\}$ mientras los $s_1 \dots s_{i-1}$ sean anulables; contiene ε solo si toda la cadena es anulable.
3. `closure(s)` calcula la cerradura de un conjunto de ítems LR(1).
4. `goto(s, x)` calcula el conjunto de ítems alcanzado desde `s` al reconocer `x`.
5. `coleccionCanonica()` realiza la construcción de $CC = \{cc_0, cc_1, \dots\}$ y del registro de transiciones.
6. `llenarTablas()` realiza la construcción de `Action` y `Goto` con detección de conflictos.

Un ítem es una tripleta $[A \to \beta \bullet \gamma,\ a]$ formada por producción, posición del punto y símbolo de anticipación. Debe representarse como tres enteros, y los conjuntos de ítems deben poder compararse para decidir si un cc ya existe en la colección.

Para que las salidas sean comparables entre equipos, los cc_i se procesan en orden de creación y, dentro de cada uno, las transiciones se calculan siguiendo el orden de primera aparición de los símbolos en la gramática, sin contar el símbolo inicial.

Llenado de tablas y conflictos

Para cada cc_i y cada ítem en él se aplican las siguientes reglas.

- $[A \to \beta \bullet c\,\gamma,\ a]$ con c terminal y $goto(cc_i, c) = cc_j$, entonces Action$[i, c]$ = shift j.
- $[A \to \beta \bullet,\ a]$ con $A \neq$ Goal, entonces Action$[i, a]$ = reduce $A \to \beta$.

- $[\text{Goal} \to S\bullet,\ \text{eof}]$, entonces Action$[i, \text{eof}] =$ accept.
- Para cada no terminal N con $goto(cc_i, N) = cc_j$, entonces Goto$[i, N] = j$.

Si una celda de `Action` recibe dos entradas distintas, la herramienta no elige una. Reporta el conflicto (shift/reduce o reduce/reduce) indicando el estado, el terminal y los ítems que lo causan, y termina con código de salida distinto de cero.

Salida

La forma de mostrar la salida la decide cada equipo. La herramienta debe mostrar las producciones numeradas, los conjuntos FIRST de todos los símbolos, cada cc_i con sus ítems y sus transiciones, y las tablas `Action` y `Goto`.

Caso de prueba de referencia

Para la gramática de paréntesis anterior, la colección canónica tiene 12 conjuntos y la tabla debe ser exactamente la siguiente, con las producciones numeradas (1) $List \to List\ Pair$, (2) $List \to Pair$, (3) $Pair \to (\ Pair\ )$, (4) $Pair \to (\ )$.

	Action			Goto	
Estado	eof	(	)	List	Pair
0		s3		1	2
1	acc	s3			4
2	r2	r2			
3		s6	s7		5
4	r1	r1			
5			s8		
6		s6	s10		9
7	r4	r4			
8	r3	r3			
9			s11		
10			r4		
11			r3		

Entregables

- Código fuente en el repositorio del equipo, compilable con `make`.
- Batería de pruebas propia con gramáticas con su tabla esperada, incluyendo al menos una gramática con conflicto shift/reduce, una con conflicto reduce/reduce y una con producciones vacías.
- Bitácora de una página.